CSE/DSC 234 Project 2 Schema Linking with Supervised Fine-Tuning

Fengfei Yu (A69042700)

Abstract

This report presents a schema-linking system for natural-language questions over relational databases. The task input is a question and a database schema, and the output is a set of tables and columns that the corresponding SQL query should touch. I trained LoRA adapters on the released labeled examples and evaluated each run with the released validation scorer. The final system uses Qwen2.5-Coder 1.5B, completion-only supervised fine-tuning, compact schema prompt, mild database balancing, and deterministic JSON post-processing. The experiments show a clear progression from a weak no-SFT baseline to the final pruned and balanced coder-model setup. The best development run scored 0.6869 overall, with a table score of 0.7753 and a column score of 0.5985.

1 Training Data Methodology

The released training data contains 301 labeled examples. Each example gives a natural-language question, a database identifier, a gold SQL query, and the table and column links extracted from that SQL query. The labels provide the supervision signal. Given a question and schema, the model should return the schema elements that the SQL answer would need. I kept the released validation set separate and used it only for scoring and model selection.

The schema files provide the database structure used by the prompts. For each database, they list the tables, columns, column types, primary keys, and foreign-key relationships. During preprocessing, I normalized every target link against these schema files so that the output uses the original table and column spelling. This matters because the scorer compares schema identifiers, not free-form natural language descriptions.

Each supervised example becomes one prompt and one JSON target. The prompt contains the task instruction, the question, and a schema view for the database. The target is a dictionary from table names to linked columns, with an empty list for table-only references. During fine-tuning, the question and schema are treated as context. Only the JSON target contributes to the training loss, which makes the optimization match the final prediction task more closely than training the model to reproduce the whole prompt.

The dataset itself is small and uneven across databases. Eight SAP Business One modules have 6 examples each, Human Resources has 12, six standalone databases have 24 each, NYSED has 37, and NTSB has 60. This imbalance is not part of the label definition; it motivated a separate training choice that tests whether mild upsampling of low-resource databases would improve validation performance without discarding larger-database examples.

2 Final Pipeline Architecture

The final inference system runs locally and does not call external APIs. It loads the public Qwen2.5-Coder-1.5B-Instruct base model and attaches the submitted adapter from the repository. This keeps the deployed artifact small while staying within the project limit of at most 2B base-model parameters.

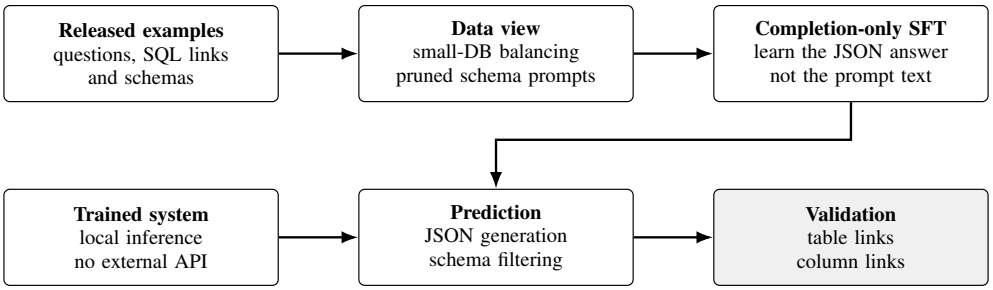


Figure 1: Overview of the final schema-linking pipeline. The top row shows how the released examples are turned into balanced, pruned training prompts for completion-only SFT. The bottom row shows local inference, where the trained adapter generates JSON links that are filtered against the schema before validation scoring.

Model and adapter. The final run uses Qwen2.5-Coder-1.5B-Instruct rather than the general Qwen2.5 1.5B instruction model. Schema linking is close to code and SQL work. The model must preserve exact identifiers, reason about joins and filters, and output a strict object. The coder base model gave better table and column scores under the same data setup [3]. I trained a LoRA adapter instead of updating the full model, which kept the submitted artifact small and made training practical on the available GPU slice [4, 5].

Hyperparameters. The best adapter uses rank 16, alpha 32, and LoRA dropout 0.05. Training used 4-bit loading, BF16 compute, batch size 1, gradient accumulation 8, cosine scheduling, 5% warmup, maximum length 8192, learning rate 8.5×10^{-5}, and 7 epochs. Rank 32 and a slightly higher learning rate both scored worse, so the final setting favors a moderate adapter with a conservative update size.

Data balancing. Upsampling is a trick in the best run, not part of the original data. I repeated only databases with fewer than 12 examples until they reached 12 rows. This changed the training file from 301 to 349 rows and mainly helped small SAP modules receive enough updates. I did not downsample NTSB or NYSED because their examples are not duplicates; they cover real query and schema patterns.

Prompt building. The final prompt uses a pruned schema view instead of the full database schema. Candidate schema items are selected by question overlap, simple synonym handling, and link statistics from the training split. For training rows, gold links are always included in the prompt so the model is not asked to predict a hidden identifier. For validation and test rows, the same pruning logic runs without gold answers. The final cap is 14 tables and 320 columns.

```
System instruction - Return only JSON schema links for the question.
Question - Which customers made orders in 2020?
Schema - Customers(CustomerID, Name); Orders(OrderID, CustomerID,
OrderDate).
Answer format - {"Customers":  ["CustomerID"], "Orders":  ["CustomerID",
"OrderDate"]}
```

Decoding and validation. Generation is greedy and deterministic. The predicted text is parsed as JSON, duplicate columns are collapsed, invalid tables or columns are removed, and valid names are restored to the original schema casing. This is a conservative validity filter. It improves precision by removing identifiers that cannot be scored as correct, but it does not add missing tables or columns.

3 Experimentation Methodology

I used RapidFire AI to run and compare supervised fine-tuning configurations. The evaluation loop was kept fixed across runs. I trained an adapter, generated predictions on the held-out development questions, and scored them with the released set-based evaluator. The table score is the average of macro precision, recall, and F1 over table sets. The column score is computed the same way over table-column pairs, and the overall score averages those two scores.

The search was organized as a sequence of controlled decisions rather than a single random sweep. **First**, I measured the base model without fine-tuning to understand the prompt-only floor. **Second**,

I added supervised fine-tuning and compared loss formatting, which made completion-only training the default. **Third**, I changed the schema view from full schema to pruned schema because the main failure pattern was extra columns. **Fourth**, I tested mild database balancing because several databases had only 6 examples. **Finally**, I switched to a coder base model and tuned the learning rate, epoch count, and adapter size around the best family. This order made each improvement interpretable because every stage answered one question before moving to the next.

Table 1: Representative configurations on the released validation split. Each row changes at least one training, data, or model choice.

Run	Base model	Data and schema setting	Training setting	Table	Column	Overall
No SFT	Qwen2.5 1.5B	full schema	no adapter	0.4113	0.2579	0.3346
Completion only	Qwen2.5 1.5B	full schema	rank 16, alpha 16, 2 epochs, lr $2e{-4}$	0.6523	0.4729	0.5626
Pruned schema	Qwen2.5 1.5B	pruned schema	rank 16, alpha 16, 4 epochs, lr $2e{-4}$	0.7049	0.5644	0.6347
Balanced to 18	Qwen2.5 1.5B	pruned, database target 18	rank 16, alpha 16, 4 epochs	0.7249	0.5511	0.6380
Balanced to 12	Qwen2.5 1.5B	pruned, database target 12	rank 16, alpha 16, 4 epochs	0.7254	0.5819	0.6536
Coder model	Qwen2.5-Coder 1.5B	pruned, database target 12	rank 16, alpha 32, 6 epochs, lr $1e{-4}$	0.7524	0.5945	0.6735
Lower learning rate	Qwen2.5-Coder 1.5B	pruned, database target 12	7 epochs, lr $7.5e{-5}$	0.7621	0.5952	0.6787
Final run	Qwen2.5-Coder 1.5B	pruned, database target 12	7 epochs, lr $8.5e{-5}$	**0.7753**	**0.5985**	**0.6869**
Higher learning rate	Qwen2.5-Coder 1.5B	pruned, database target 12	7 epochs, lr $9e{-5}$	0.7351	0.5711	0.6531
Larger adapter	Qwen2.5-Coder 1.5B	pruned, database target 12	rank 32, alpha 64, 7 epochs	0.7530	0.5916	0.6723
Complex-row repeat	Qwen2.5 1.5B	repeat rows with many links	rank 16, alpha 16, 4 epochs	0.7393	0.5880	0.6636
Alternative coder	DeepSeek-Coder 1.3B	pruned, database target 12	same final recipe	0.7138	0.4831	0.5984

Table 2: Metric breakdown for the main improvement path. Table and column scores are each the average of their precision, recall, and F1 columns.

Run	P_T	R_T	$F1_T$	Table	P_C	R_C	$F1_C$	Column	Overall
No SFT	0.3230	0.5553	0.3556	0.4113	0.3079	0.2479	0.2180	0.2579	0.3346
Completion only	0.6832	0.6361	0.6375	0.6523	0.5066	0.4632	0.4488	0.4729	0.5626
Pruned schema	0.7153	0.7112	0.6882	0.7049	0.5845	0.5688	0.5400	0.5644	0.6347
Balanced to 12	0.7231	0.7450	0.7079	0.7254	0.5921	0.5978	0.5558	0.5819	0.6536
Coder model	0.7607	0.7591	0.7375	0.7524	0.6018	0.6092	0.5725	0.5945	0.6735
Final run	**0.7812**	**0.7847**	**0.7600**	**0.7753**	**0.6095**	**0.6125**	**0.5734**	**0.5985**	**0.6869**

Tables 1 and 2 show the path to the final run. Each stage fixed a concrete weakness from the previous stage, moving from output format to schema noise, data imbalance, base-model fit, and then optimization. Stronger database balancing, a larger adapter, and a higher learning rate all lowered the combined score, so they were not used in the final system.

This experimental path also explains why the final pipeline is conservative. Full schemas gave the model too many plausible columns to copy. Heavy repetition of small databases improved some table choices but hurt column stability. The chosen pipeline is therefore not the largest or most complicated one I tried. It is the setup where each component improved the held-out score without creating a larger regression elsewhere.

4 Results and Analysis

Base-model performance. The prompt-only baseline scored 0.3346 overall. It often produced extra tables, invalid JSON, or plausible columns under the wrong table. This result was useful because it showed that the model had some prior SQL knowledge, but not enough task discipline to solve schema linking reliably without training. Its table recall was 0.5553, much higher than its table precision of 0.3230, which means the base model often found related tables but returned too many of them.

Effect of completion-only SFT. Completion-only fine-tuning raised the score to 0.5626. The improvement came from teaching the model to emit the answer object directly instead of explaining itself or copying schema text. Masking the prompt tokens also made the small dataset more useful, because every loss-bearing token belonged to the required JSON answer. The table score improved by 0.2410 and the column score improved by 0.2150 over the no-SFT run, so this was the largest single jump in the early experiments.

Effect of schema pruning. Pruning the schema raised the Qwen2.5 run to 0.6347. This change helped mostly at the column level. Full-schema prompts gave the model many reasonable-looking

but wrong columns; the pruned prompt reduced those distractors while still keeping the gold identifiers visible during training. Column precision rose from 0.5066 to 0.5845, and column F1 rose from 0.4488 to 0.5400.

Effect of data balancing. Upsampling databases with fewer than 12 examples improved the pruned Qwen2.5 run from 0.6347 to 0.6536. A stronger target of 18 improved table coverage but reduced column score, so the final system uses balancing as a gentle correction rather than forcing every database to appear equally often. The target-12 run mainly improved recall, with table recall moving from 0.7112 to 0.7450 and column recall from 0.5688 to 0.5978. I avoided downsampling because the large-database rows are distinct examples, not duplicate copies.

Effect of the coder base model. Switching to Qwen2.5-Coder-1.5B improved both metrics and reached 0.6735 before the final learning-rate search. This was the largest later improvement. It fits the task because schema linking requires exact identifier handling, SQL-style reasoning, and a strict output format, all of which are closer to a code model's training distribution than to a general chat model's. The coder model improved table precision by 0.0376 over the balanced Qwen2.5 run and kept column recall above 0.60.

Optimization results. The final local search found that 7 epochs and learning rate 8.5×10^{-5} worked best for the coder model, reaching 0.6869 overall. A slightly lower rate was close but weaker, a slightly higher rate regressed, and rank 32 did not help. This suggests that the final error is not simply adapter capacity; it is more about learning stable column decisions from a small and uneven dataset. Compared with the first coder run, the final setting gained 0.0229 table score but only 0.0040 column score.

Remaining errors. Table prediction is stronger than column prediction. Many questions reveal the correct table from surface words, but the right column depends on a measure, condition, date field, or join path. The final post-processing can remove invalid identifiers, but it cannot recover a missed measure column or infer a join key that the model failed to output. In the final run, table score was 0.7753 while column score was 0.5985, so most remaining room is in fine-grained column selection.

Main takeaway. The best result came from aligning the training target, schema context, data distribution, base model, and optimization schedule. More context, more repetition, or a larger adapter did not automatically help. The final model is the setting where those choices improved the combined validation score without causing a larger regression in table or column behavior.

5 AI Tools Statement

AI tools were used as coding, debugging, experiment-organization, and writing aids. They helped with inspecting project files, organizing commands, summarizing results for experiment runs, and editing report/README wording. The experiments, scores, and submitted adapter come from local project runs and the released evaluator.

References

[1] Kyle Luoma and Abhinav Kumar. SNAILS: Schema Naming Assessments for Improved LLM-Based SQL Inference. Proc. ACM SIGMOD, 2025.

[2] RapidFire AI. RapidFire AI. `https://www.rapidfire.ai/`.

[3] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-Coder technical report. arXiv:2409.12186, 2024.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. arXiv:2106.09685, 2021.

[5] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. arXiv:2305.14314, 2023.